

Conclusiones Taller Depuración de Código.

Ejercicio 1 → Análisis de Datos con Puntos

El error muestra la importancia del manejo de excepciones cuando se trabaja con archivos externos, pues su existencia y disponibilidad no dependen del programa.

Usar try/except nos ayuda a anticipar fallos y responder de forma controlada en lugar de dejar que el programa se detenga, aplicando lo que se conoce en desarrollo como programación defensiva, es decir, anticipar los posibles puntos de falla.

Esto es especialmente relevante en los flujos de análisis de datos, donde los archivos suelen estar en fuentes externas y también pueden cambiar constantemente.

Ejercicio 2 → Función recursiva

Toda función recursiva necesita un caso base que pueda detener las llamadas sucesivas, sin ese caso base el programa entra en una pila de llamadas infinitas agotando la memoria disponible.

El error que lanza es un (recursionerror) El código es válido para el intérprete, es por ello que no es un error sintáctico. Es semántico, puesto que la lógica del algoritmo está incompleta.

Es importante comprender que la recursión exige que cada llamada se acerque de forma progresiva a una condición de parada, un claro ejemplo es la definición matemática del factorial.

Ejercicio 3 → Gráfico de una Distribución con Matplotlib

Aunque no se genera un error en la ejecución, la visualización de los datos es confusa.

Definir de forma explícita los bins, colores, títulos y etiquetas más que un detalle estético, es algo que determina si el histograma comunica de forma correcta la distribución de los datos, o por el contrario, oculta información relevante.

Un código que funciona no siempre puede considerarse como un código correcto.

Ejercicio 4 → Web Scraping con BeautifulSoup

En este ejercicio se expone un limitante que no depende del código, sino del protocolo HTTP y las normativas de seguridad de los servidores.

Muchos sitios bloquean solicitudes sin User-Agent, por ser señal común de bot. Al incluir un encabezado que simule un navegador real llevamos a la práctica el estándar en web scraping, aunque debe respetar siempre los términos de servicio de cada sitio consultado.

Ejercicio 5 → Manipulación de Arrays con Numpy

El error que nos lanza (ValueError) cuando usamos reshape() nos muestra que el número total de elementos debe conservarse en cualquier transformación de forma, puesto que un array es básicamente un bloque de memoria interpretado respecto a ciertas dimensiones.

Verificar que tanto filas como columnas coincidan con el total de elementos es una habilidad esencial en computación científica, y este es un tipo de error que es muy frecuente en machine learning y el procesamiento de imágenes.

Ejercicio 6 → Uso de hilos con threading

En el ejercicio se aborda una condición de carrera (race condition), donde dos hilos modifican de forma simultánea una misma variable, pero no tienen ningún mecanismo de sincronización, es así como se generan resultados inconsistentes, y estos no siempre se detectan fácilmente.

La solución mediante un objeto Lock nos garantiza una exclusión mutua, es decir, nos asegura que un solo hilo a la vez ejecute la sección crítica. Este concepto es la base de la programación concurrente segura.

Ejercicio 7 → API con request

La ausencia de manejo de errores HTTP provoca que toda la comunicación con una API dependa de factores externos, como la disponibilidad del servidor, red, límites de uso, todo esto fuera del control del programador.

Un código robusto no debe asumir que la respuesta siempre sea exitosa, primero debe verificar el estado de la respuesta, establecer un timeout y capturar las excepciones específicas.

Es una práctica esencial en el desarrollo profesional, donde los fallos de red son inevitables.

Ejercicio 8 → Clase con Herencia

Cuando una subclase sobrescribe `init`, deja de heredar automáticamente el constructor de la clase base a menos que este sea invocado con `super`. Cuando se omite esta llamada se rompe la cadena de inicialización provocando que atributos esenciales, como nombre, nunca pueden definirse, generando un `AttributeError`.

La herencia no es automática en todos sus aspectos, es por ello que el programador debe garantizar la inicialización completa.

Ejercicio 9 → Conexión a Base de Datos SQLite

Se evidencia que las operaciones de modificación en la base de datos ocurren dentro de una transacción que no se hace permanente hasta ejecutar el `commit()`. Si la conexión se cierra sin confirmar los cambios, estos se van a perder sin poder generar un error visible, en entornos reales es un fallo riesgoso y comprometedor. Como nunca se llama `commit`, nunca se confirma nada, y como basados en el principio de atomicidad de las bases de datos una transacción es todo o nada la falla no será detectable, entonces no se garantiza su persistencia.

Ejercicio 10 → Algoritmo de Ordenamiento

Aunque el algoritmo original sí logra ordenar la lista, no reduce el rango de comparación en cada pasada, generando comparaciones redundantes sobre elementos ya ordenados. Aquí se detecta un error funcional, de un error de eficiencia, este último afecta directamente en el rendimiento. Ajustar el rango interno nos ayuda a optimizar la ejecución real del algoritmo.